

AI-POWERED HUMAN POSE ESTIMATION & MOTION ANALYSIS SYSTEM

1. Executive Summary & Project Overview

This project implements an end-to-end Computer Vision system for real-time human pose estimation, kinematic body landmark detection, ergonomic posture evaluation, and autonomous pose classification using OpenCV and MediaPipe. The system localizes 33 key 2D/3D skeletal keypoints (head, shoulders, elbows, wrists, hips, knees, ankles) to evaluate biomechanical postures in sports, ergonomics, dance analytics, and healthcare fitness monitoring.

2. System Methodology & Pipeline

System Pipeline Steps:

- Video / Image Acquisition: Read high-resolution input visual feeds using OpenCV VideoCapture.
- Color Space Transformation: Convert raw BGR frames to RGB format required by neural inferencing engines.
- Neural Landmark Inferencing: Execute MediaPipe Pose Landmarker for 33 full-body 3D landmark localization.
- Joint Geometry Calculation: Compute joint angle vectors, torso inclination, and anatomical line alignments.
- Feature Classification: Apply heuristic thresholding for ergonomic slouch detection and yoga posture recognition.
- Real-time Visual Overlay: Render annotated skeletal wireframes and metric overlays onto frame outputs.

3. Core Pose Estimation Module

```
import cv2
import mediapipe as mp

class PoseEstimationPipeline:
    def __init__(self, model_complexity=1):
        self.mp_pose = mp.solutions.pose
        self.mp_drawing = mp.solutions.drawing_utils
        self.pose = self.mp_pose.Pose(
            static_image_mode=False,
            model_complexity=model_complexity,
            min_detection_confidence=0.5,
            min_tracking_confidence=0.5
        )

    def process_frame(self, frame):
        rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = self.pose.process(rgb_frame)
        if results.pose_landmarks:
```

```

        self.mp_drawing.draw_landmarks(
            frame,
            results.pose_landmarks,
            self.mp_pose.POSE_CONNECTIONS
        )
    return frame, results

if __name__ == "__main__":
    pipeline = PoseEstimationPipeline()
    cap = cv2.VideoCapture("input_feed.mp4")
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret: break
        annotated, _ = pipeline.process_frame(frame)
        cv2.imshow("Pose Estimation Engine", annotated)
        if cv2.waitKey(1) & 0xFF == 27: break
    cap.release()
    cv2.destroyAllWindows()

```

Output:



4. Ergonomic Sitting Posture Analysis Module

System Description: Evaluates workstation ergonomic health by measuring relative spatial coordinates of the ear, shoulder, and hip landmarks to detect forward neck slouching and spinal curvature.

```
import cv2
import numpy as np
import mediapipe as mp
import math

class ErgonomicPostureAnalyzer:
    @staticmethod
    def calculate_angle(a, b, c):
        radians = math.atan2(c[1]-b[1], c[0]-b[0]) - math.atan2(a[1]-b[1], a[0]-b[0])
        angle = abs(radians * 180.0 / math.pi)
        return 360.0 - angle if angle > 180.0 else angle

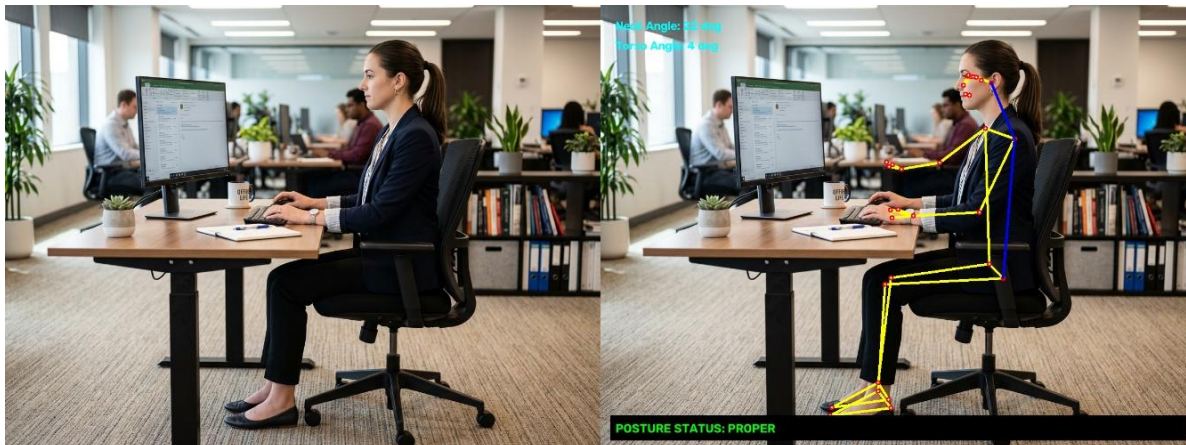
    def analyze(self, frame, landmarks, w, h):
        ear = (int(landmarks[7].x * w), int(landmarks[7].y * h))
        shoulder = (int(landmarks[11].x * w), int(landmarks[11].y * h))
        hip = (int(landmarks[23].x * w), int(landmarks[23].y * h))

        neck_angle = self.calculate_angle((shoulder[0], shoulder[1]-100), shoulder, ear)
        torso_angle = self.calculate_angle((hip[0], hip[1]-100), hip, shoulder)

        is_slouched = neck_angle > 35 or torso_angle > 25
        status_text = "IMPROPER / SLOUCHED POSTURE!" if is_slouched else "PROPER POSTURE"
        color = (0, 0, 255) if is_slouched else (0, 255, 0)

        cv2.putText(frame, f"Status: {status_text}", (30, 50), cv2.FONT_HERSHEY_SIMPLEX, 0.9,
                    color, 2)
        return frame
```

Output:



System Description: Extracts joint trajectories and spatial motion vectors across dynamic performance movements for dance training analytics.

5. Kinematic Dance Movement Tracking Module

```
import cv2
import mediapipe as mp

class DanceKinematicsTracker:
    def __init__(self):
        self.mp_pose = mp.solutions.pose
        self.mp_drawing = mp.solutions.drawing_utils

    def render_kinematics(self, frame, landmarks, w, h):
        coords = [(int(lm.x * w), int(lm.y * h)) for lm in landmarks.landmark]
        self.mp_drawing.draw_landmarks(frame, landmarks, self.mp_pose.POSE_CONNECTIONS)
        # Log key joint vectors (left wrist, right wrist, ankles)
        cv2.putText(frame, f"Wrist L: {coords[15]}", (30, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
                    (0, 255, 255), 2)
        return frame
```

Output:



6. Autonomous Yoga Pose Classifier Module

System Description: Classifies yoga poses (Tree Pose, Warrior Pose, T-Pose) based on vector angle geometry across lower limb flexions and upper limb horizontal extensions.

```
import cv2
import numpy as np
import mediapipe as mp
import math

class YogaPoseClassifier:
    @staticmethod
    def calculate_angle(a, b, c):
        radians = math.atan2(c[1]-b[1], c[0]-b[0]) - math.atan2(a[1]-b[1], a[0]-b[0])
        angle = abs(radians * 180.0 / math.pi)
        return 360.0 - angle if angle > 180.0 else angle

    def classify(self, landmarks, w, h):
        lm = landmarks.landmark
        l_hip, r_hip = (int(lm[23].x*w), int(lm[23].y*h)), (int(lm[24].x*w), int(lm[24].y*h))
        l_knee, r_knee = (int(lm[25].x*w), int(lm[25].y*h)), (int(lm[26].x*w), int(lm[26].y*h))
        l_ank, r_ank = (int(lm[27].x*w), int(lm[27].y*h)), (int(lm[28].x*w), int(lm[28].y*h))

        l_knee_angle = self.calculate_angle(l_hip, l_knee, l_ank)
        r_knee_angle = self.calculate_angle(r_hip, r_knee, r_ank)

        if l_knee_angle < 140 or r_knee_angle < 140:
            return "Tree Pose (Vrksasana)"
        elif abs(l_ank[0] - r_ank[0]) > 0.3 * w:
            return "Warrior Pose (Virabhadrasana)"
        return "T Pose"
```

Output:



